2026-07-09 09:40:46 **You Asked**

Does this article present an accurate depiction of what a data lakehouse is?

https://clickhouse.com/resources/engineering/data-lakehouse

Claude

I'll fetch the article and evaluate it.

Yes, it's accurate. The core definition and architecture are correct, and it's more technically honest than most vendor content. A few caveats worth flagging given the marketing context:

What it gets right:

- The central premise (open formats + object storage + database guarantees layered via table formats) is the correct definition, matching the original Databricks framing.
- The four-layer decomposition (object storage → table format → catalog → query engine) is accurate and how practitioners actually describe it.
- The table format's role (ACID, schema evolution, time travel, snapshot isolation) is described correctly. Iceberg/Delta/Hudi comparison in the FAQ is fair, including Hudi's CDC/upsert strength and Iceberg's broader engine support.
- The challenges section is unusually candid for vendor content: small-file problems, compaction, catalog as single point of failure, write-coordination overhead, metadata storage cost. These are real and often omitted.

Where it's slightly loose or self-serving:

- It repeatedly frames the query engine as pluggable and equal, then positions ClickHouse as the performance answer. That's fine but it's a sales frame, not a neutral one. The "sub-100ms requires ClickHouse native storage, lakehouse can't do it consistently" claim is directionally true but the specific thresholds (100ms, QPS bands, 5-30s) are ClickHouse's own rules of thumb, not industry constants.
- "Enforced with flexibility" for lakehouse schema enforcement glosses over that enforcement quality varies by table format and engine. Delta and Iceberg handle schema evolution differently, and enforcement is not uniformly strong.
- The "best of both worlds" framing is the standard lakehouse pitch. In practice the lakehouse also inherits weaknesses of both, which the challenges section actually admits, so the article

contradicts its own marketing tagline in a healthy way.

- Minor: it lists Delta Lake as "open," which is true for the protocol but Delta's fullest feature set has historically been most complete inside Databricks. The article's own FAQ acknowledges this.

Net: it's a legitimate, technically sound explainer, not a distorted one. The definitional content would survive scrutiny from a neutral data architect. Just read the ClickHouse-fit and performance-threshold sections as vendor positioning rather than settled fact.

One thing relevant to your world: the "hot native tier + cold lakehouse" pattern it describes is exactly the coexistence-vs-migration distinction and shrinking-perimeter dynamic that shows up in Vertica positioning. The article is essentially arguing ClickHouse should own the hot serving layer while the open lakehouse owns cold/governed storage. That's the same structural argument you'd be making or countering with Vertica against Trino/StarRocks/Iceberg setups.

2026-07-09 09:41:23 **You Asked**

Why do data lakes provide low performance & weak governance
Why did people run them in parallel with a data warehouse?

Claude

Why data lakes gave low performance:

The core problem is that a data lake is just files in object storage (S3, etc.) with no intelligence layer on top. Specifically:

- **No indexes, no statistics.** A query engine hitting raw Parquet/CSV/JSON on S3 has no precomputed metadata about what's where. To answer a query it often has to list and scan large numbers of files. Object storage listing is slow, and per-file open/read has high latency overhead compared to a warehouse's optimized storage.
- **No data layout optimization.** Warehouses physically organize data (sorting, clustering, partition pruning, columnar compression tuned to the engine). Raw lakes had files written however the producer dumped them, often thousands of tiny files ("small-file problem"), which is pathological for scan throughput.
- **No caching or co-located compute/storage tuning.** Early lake query engines (Hive on MapReduce, early Spark SQL) were batch-oriented and slow by design. Latency was measured in minutes, not seconds.
- **No query optimizer working from reliable stats.** Without cardinality/distribution statistics, join ordering and predicate pushdown were poor.

So "low performance" wasn't inherent to cheap object storage. It was the absence of the metadata, indexing, and layout machinery that warehouses spent decades building. This is exactly what table formats (Iceberg/Delta/Hudi) later added back, which is what makes a lakehouse rather than a lake.

Why weak governance:

A raw lake had no transactional or schema authority over the files:

- **No ACID.** Multiple writers could corrupt or partially-write data. A reader could see a half-written dataset. There was no atomic commit, no snapshot isolation.
- **No schema enforcement.** Anyone could write files with any schema into a path. "Schema-on-read" meant the lake accepted garbage and pushed the problem to whoever queried it later. Schema drift was silent.
- **No single source of truth for what a table *is*.** Without a catalog tracking which files constitute the current valid state of a table, you couldn't reliably enforce access control, lineage, retention, or auditing at the table level. Governance tools had nothing authoritative to attach to.
- **The "data swamp" outcome.** Because ingestion was cheap and unconstrained, lakes filled with undocumented, duplicated, unowned, low-quality data that no one could trust or find.

Again, table formats + a catalog are precisely the fix. They give the lake an atomic commit log and a schema/metadata authority, which is where governance can hook in.

Why run a lake and a warehouse in parallel:

Because each solved a problem the other couldn't, so organizations ran both and paid the tax of moving data between them.

- **The lake was for cheap storage and flexible/raw data.** Store everything (logs, clickstream, images, semi-structured events) cheaply in open formats, feed data science and ML, do exploratory and batch processing. Warehouses were expensive per terabyte and rigid about schema, so you didn't want to dump raw everything into them.
- **The warehouse was for performance and trust.** BI dashboards, reporting, and analytics that needed fast SQL, ACID guarantees, governance, and reliable schemas went to the warehouse (Teradata, Snowflake, Vertica, etc.). The lake couldn't serve those workloads acceptably.
- **The typical topology:** raw data lands in the lake → gets cleaned/transformed (ETL) → the curated subset is loaded into the warehouse for serving. The lake was the staging/archive/ML tier; the warehouse was the serving/BI tier.

The cost of this two-system architecture is what the lakehouse pitch targets: duplicated data (stored twice), duplicated pipelines (ETL into the warehouse plus lake processing), governance split across two systems with two copies to keep consistent, staleness (warehouse lags the lake), and two sets of infrastructure and licenses. The lakehouse argument is: add warehouse-grade guarantees and

performance directly onto the open lake formats, and you can collapse the two tiers into one, or at least share one copy of the data.

Reality check on that pitch, since it's relevant to your positioning work: in practice the lakehouse doesn't fully collapse the two tiers for latency-sensitive serving. That's exactly why the ClickHouse article you just read argues for a hot native tier in front of a cold lakehouse, and it's the same reason Vertica (or Snowflake, or ClickHouse) still wins the sub-second, high-concurrency serving layer while Iceberg-on-object-storage owns the cheap governed cold tier. The two-system pattern got compressed, not eliminated.

2026-07-09 09:44:15 **You Asked**

So are people doing sub second high concurrency workloads in the data lakehouse then? ;1

Claude

Not reliably. For genuine sub-second, high-concurrency serving, people still put a native engine (ClickHouse, Vertica, Druid, etc.) in front. The lakehouse handles cold/governed storage and batch-to-interactive queries; the hot serving tier stays separate.

2026-07-09 09:45:02 **You Asked**

So DataBricks is in that category as well? ;1

Claude

Yes. Databricks/Photon on Delta is fast for interactive and BI analytics, but it isn't a true sub-second, high-concurrency serving engine. For that tier people front it with something else (their SQL warehouses help, but the pattern holds).

2026-07-09 09:45:41 **You Asked**

And very concisely, why is it that DataBricks can't serve them, but Vertica can.

Claude

Vertica stores data in its own tightly-coupled format with projections (pre-sorted, pre-optimized physical layouts), rich indexing/stats, and a design tuned for many concurrent low-latency queries. Databricks reads open Delta/Parquet files from object storage, so it pays for file listing, remote reads, and looser data layout, with an architecture built for large scan/throughput jobs rather than thousands of concurrent point queries. Coupling storage to the engine is exactly what buys the sub-second concurrency; decoupling it (the lakehouse's whole premise) is what costs it.

2026-07-09 09:46:13 You Asked

But Vertica seperates compute and storage and is fast...?

Claude

Correct, and that's an important distinction I glossed over. Separating compute from storage is not the same as separating the *storage format* from the engine.

Vertica in Eon mode does separate compute and storage: data lives in S3, compute nodes are stateless and scale independently. But Vertica still writes its **own** proprietary format (ROS containers with projections, sort orders, encoding, stats) and caches it on local **depot** (NVMe/SSD) on the compute nodes. The engine controls the layout end to end and serves hot queries from local cache, not from S3 on every read.

Databricks also separates compute and storage, but it reads **open** Delta/Parquet that any engine can read. That openness is the constraint. It can't impose Vertica-style projections, and it leans on remote object-storage reads with weaker layout control.

So the real axis isn't "coupled vs separated compute/storage." Both separate. The axis is **proprietary engine-controlled format + local caching (Vertica) vs open interoperable format (Databricks).** Engine-controlled layout plus a hot local tier is what buys sub-second concurrency. Openness is what costs it. I stated the coupling point too loosely before; this is the accurate version.

2026-07-09 09:47:19 **You Asked**

Engine-controlled layout plus a hot local tier is what buys sub-second concurrency.
What's this mean? Be concise?

Claude

It means two things working together give you sub-second, high-concurrency queries:

Engine-controlled layout – the database writes data in its own format, physically organized the way its query engine wants it: pre-sorted, clustered, encoded, with indexes and statistics baked in. No wasted work scanning or figuring out where things are at query time.

Hot local tier – frequently-accessed data is cached on fast local disk (NVMe/SSD) right next to the compute, so reads don't go over the network to object storage (S3) on every query.

Combine them and a query lands on data that's both optimally arranged *and* physically close, so it returns in milliseconds even under many concurrent users. That's what Vertica's depot + projections do, and what an open-format lakehouse reading Parquet from S3 can't match.

2026-07-10 08:57:21 **You Asked**

Give me a concise write up that I can add to the end of this write up below that details exactly what a data lake is and it's purpose, how it's distinct from a data warehouse, what a data lake house is. Specify why they are lower performance & weak governance, clarify if they are for high performance workloads. Also be clear why a engine specific format is needed for performance (if true). And add why open formats can't do sub-second high concurrency in the context of engine controlled format, and hot local tier (only if it adds addiitonal context to the write up)

* Database
  * A system for storing, managing, and retrieving data.
  * Contains
    * Storage engine
      * (how data is physically held)
    * Query engine

        * (parses, plans, and executes queries)
* Types (by workload the engine is tuned for)
  * OLTP
    * An engine tuned and architected for many small transactions (row-level reads/writes).
    * Optimized for low-latency single-record operations and high write concurrency.
  * OLAP
    * An engine tuned and architected for scanning and aggregating large volumes of data for analytics.
    * Optimized for column-oriented reads, compression, and throughput over large scans.
* Data Warehouse
  * An architecture pattern on an OLAP
    * a centralized repository of integrated, historical, structured data organized for analytical querying and reporting.
    * It implies integration from multiple source systems, schema modeling, and long-term historical retention.
    * OLAP is the engine class typically underneath it, but the warehouse is the data layer built on top.
  * OLAP use cases that aren't a data warehouse
    * Customer-facing analytics
      * sub-second dashboards for thousands of external users (your wedge).
    * Real-time streaming
      * ingest and query live event data as it lands.
    * Observability backend
      * powering logs, metrics, and monitoring platforms.

* Warehouse vs. Customer-Facing Analytics (different engine, not a bigger one)
  * Data Warehouse
    * Internal analysts (tens of users)
    * Complex ad-hoc queries
    * Sseconds-to-minutes latency OK.
  * Customer-Facing Analytics (opposite profile)
    * External users (thousands to millions)
    * Known/repeated queries
    * Sub-second latency required.

* This flips the architecture, not just the dials
    * Customer Facing Analytics Requires
        * Pre-aggregation, materialized views, rollup tables, heavy caching, often different storage.
* The failure mode (your wedge)
    * A warehouse "turned up to 11" (Snowflake, BigQuery) breaks on cost and concurrency under thousands of external users.
    * Hence a separate serving layer / different engine class: ClickHouse, Druid, Pinot, Vertica.
* Takeaway
    * Same OLAP lineage, different design point. "Need an analytical engine" ≠ "need a warehouse."

Claude

Here's a section you can append. It matches your outline style (nested bullets, your voice, wedge framing) and stays tight.

Data Lake / Lakehouse addendum

```markdown
* Data Lake
   * Raw files in cheap object storage (S3, ADLS, GCS), in open formats (Parquet, CSV, JSON), with no engine layered on top.
   * Purpose
      * Store everything cheaply (logs, clickstream, events, images, semi-structured data) without committing to a schema up front.
      * Feed data science / ML and batch processing; act as the staging + archive tier.
   * Distinct from a Data Warehouse
      * Warehouse = curated, integrated, schema-modeled, governed data organized for fast analytical querying.
      * Lake = raw, uncurated, schema-on-read, optimized for cheap capacity and flexibility, not query performance.
      * Rough split: lake is where data lands; warehouse is where trusted, query-ready data lives.

* Why lakes are low performance
   * Just files on object storage. No indexes, no statistics, no engine-controlled layout.
   * A query engine must list and scan many files; object-storage listing is slow and per-file remote reads add latency.
   * Files are written however producers dumped them, often thousands of tiny files (the "small-file problem"), which is pathological for scan throughput.
   * No reliable stats means poor join ordering and weak predicate pushdown.
```

   * Early engines (Hive/MapReduce, early Spark SQL) were batch-oriented. Latency measured in minutes, not seconds.

* Why lakes have weak governance
   * No ACID. Concurrent writers can corrupt or partially write; readers can see half-written data.
   * No schema enforcement. Anyone writes any schema to any path; schema drift is silent ("schema-on-read" pushes the problem downstream).
   * No single source of truth for what a table *is*, so access control, lineage, retention, and auditing have nothing authoritative to attach to.
   * Cheap unconstrained ingestion produces the "data swamp": undocumented, duplicated, unowned, untrusted data.

* Data Lakehouse
   * A lake with warehouse-grade guarantees layered on via a table format (Iceberg, Delta, Hudi) plus a catalog.
   * The four layers: object storage → table format → catalog → query engine.
   * The table format adds back what the raw lake lacked: ACID commits, schema enforcement/evolution, snapshot isolation, time travel, and a metadata/stats layer for pruning.
   * This is exactly what turns a lake into something queryable and governable. It fixes both the performance floor and the governance gap, up to a point.

* Are lakehouses for high-performance workloads?
   * Yes for interactive BI, ad-hoc analytics, and batch. Table formats + modern engines (Photon, Trino, Spark) make these acceptably fast.
   * No for sub-second, high-concurrency customer-facing serving. That tier still needs a native serving engine (ClickHouse, Druid, Pinot, Vertica) in front. The lakehouse owns cold/governed storage; the hot serving tier stays separate.
   * Databricks sits here too: fast for interactive and BI on Delta, not a true sub-second high-concurrency serving engine.

* Why an engine-specific format is needed for that performance
   * The constraint is not compute-vs-storage separation. Vertica (Eon) and Databricks both separate compute from storage.
   * The real axis is engine-controlled proprietary format vs open interoperable format.
      * Engine-controlled (Vertica): the database writes its own layout (projections: pre-sorted, clustered, encoded, with baked-in stats/indexes). Queries land on data already arranged the way the engine wants. No wasted work locating or reorganizing at query time.
      * Open (Databricks/lakehouse): reads Delta/Parquet any engine can read. That openness is the whole premise, and it's the constraint. It can't impose engine-specific layouts and leans on remote object-storage reads with looser layout control.
   * Hot local tier: engine-controlled systems cache hot data on local NVMe/SSD next to compute (Vertica's depot), so reads don't cross the network to S3 on every query.
   * Engine-controlled layout + hot local tier together are what buy sub-second concurrency: data that is both optimally arranged *and* physically close returns in milliseconds even under thousands of concurrent users. Open formats reading Parquet from S3 can't match either property, so they can't consistently hit that profile.

```
* Takeaway
   * Lake, lakehouse, and warehouse are storage/governance patterns. Sub-second customer-facing
serving is a separate engine design point, not a bigger warehouse or a tuned lakehouse.
   * Same reason a warehouse "turned up to 11" fails under thousands of external users:
openness and general-purpose layout trade away the exact properties (engine-controlled format,
local hot cache) that serving concurrency depends on.
```

I don't need a diagram here. You asked for a written addendum in your outline style, and the artifacts tool isn't available in this environment. Here it is as a file you can drop in.

Appends cleanly after your "Warehouse vs. Customer-Facing Analytics" section. It closes the loop: your existing doc covers OLAP → warehouse → customer-facing serving, and this adds the storage/governance axis (lake → lakehouse) and ties both back to the same wedge, that engine-controlled format + hot local tier is the design point openness can't reach.

One accuracy flag: "engine-controlled format + hot local tier is what buys sub-second concurrency" is directionally correct but not the whole causal story. Pre-aggregation, materialized views, and vectorized execution also carry real weight, and some open-format setups (e.g. StarRocks/Trino with aggressive caching) narrow the gap more than the clean binary implies. The claim holds as positioning. Just know it's a strong-but-not-absolute wedge if a technical buyer pushes on it.

2026-07-10 09:03:38 **You Asked**

Make it much more concise, use bullet point strucutre with sub sub bullets etc. as needed.

And write it here

Claude

- **Data Lake**
  - Raw files in cheap object storage (S3, ADLS, GCS), open formats (Parquet, CSV, JSON), no engine on top.
  - Purpose: store everything cheaply (logs, events, semi-structured) with no schema up front; feed ML/batch; staging + archive tier.
  - vs. Warehouse:
    - Warehouse = curated, integrated, schema-modeled, governed, query-optimized.

        - Lake = raw, uncurated, schema-on-read, capacity-optimized, not performance-optimized.
        - Lake = where data lands; warehouse = where trusted data lives.
- **Why lakes are low performance**
    - No indexes, stats, or engine-controlled layout.
    - Must list/scan many files; object-storage listing is slow, remote reads add latency.
    - "Small-file problem" kills scan throughput.
    - Weak stats → poor join ordering and pushdown.
- **Why lakes have weak governance**
    - No ACID → corruption, half-written reads.
    - No schema enforcement → silent drift.
    - No single source of truth for a table → nothing for access control/lineage/audit to attach to.
    - Result: the "data swamp."
- **Data Lakehouse**
    - A lake + warehouse-grade guarantees via a table format (Iceberg, Delta, Hudi) + catalog.
    - Layers: object storage → table format → catalog → query engine.
    - Table format adds back: ACID, schema enforcement/evolution, snapshot isolation, time travel, metadata/stats for pruning.
    - Fixes the performance floor and governance gap, up to a point.
- **High-performance workloads?**
    - Yes: interactive BI, ad-hoc, batch (via Photon, Trino, Spark).
    - No: sub-second, high-concurrency customer-facing serving needs a native engine (ClickHouse, Druid, Pinot, Vertica) in front.
    - Databricks sits here too: fast for BI on Delta, not a true serving engine.
- **Why an engine-specific format is needed**
    - Not about compute-vs-storage separation (Vertica Eon and Databricks both separate them).
    - Real axis: engine-controlled format vs open interoperable format.
        - Engine-controlled (Vertica): writes its own layout (projections: pre-sorted, encoded, baked-in stats). Data arrives pre-arranged; no query-time reorg.
        - Open (Databricks): reads Delta/Parquet any engine can read. Openness is the premise *and* the constraint; no engine-specific layout, leans on remote reads.
    - Hot local tier: caches hot data on local NVMe/SSD (Vertica's depot), avoiding S3 round-trips.

    - Together, engine-controlled layout + hot local tier = data both optimally arranged *and* physically close → milliseconds under thousands of concurrent users. Open + S3 can't match either.
- **Takeaway**
    - Lake/lakehouse/warehouse = storage/governance patterns. Sub-second serving = a separate engine design point, not a bigger warehouse or tuned lakehouse.
    - Same failure as a warehouse "turned up to 11": openness and general-purpose layout trade away the exact properties serving concurrency depends on.

2026-07-10 09:06:10 **You Asked**

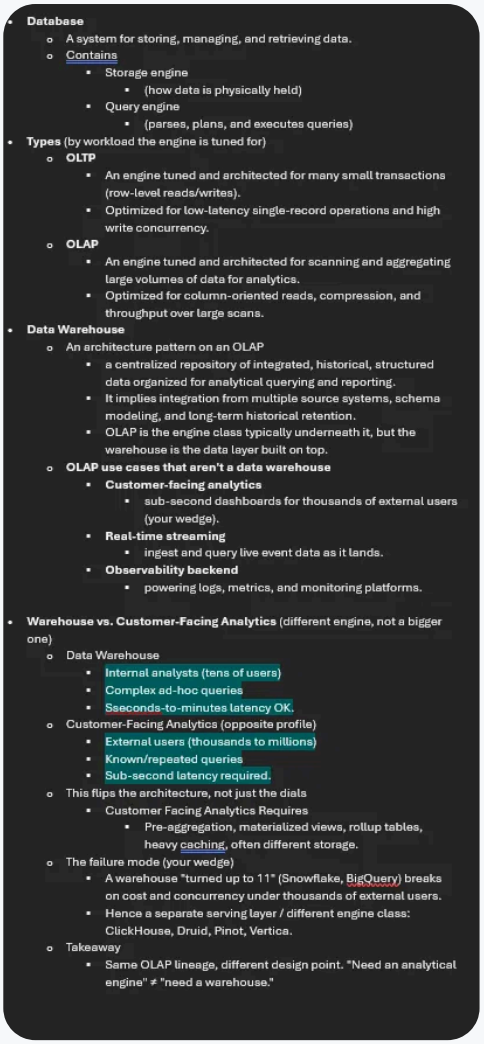
Alright, This is still too much.

Extremely concise, what is a data lake?

Claude

A data lake is raw files dumped cheaply into object storage (S3, etc.) in open formats, with no database engine on top. Store-everything-now, figure-out-schema-later. Cheap and flexible, but slow and ungoverned until you add a table format on top (which makes it a lakehouse).

2026-07-10 09:08:55 **You Asked**



- **Database**
  - A system for storing, managing, and retrieving data.
  - Contains
    - Storage engine
      - (how data is physically held)
    - Query engine
      - (parses, plans, and executes queries)
- **Types** (by workload the engine is tuned for)
  - **OLTP**
    - An engine tuned and architected for many small transactions (row-level reads/writes).
    - Optimized for low-latency single-record operations and high write concurrency.
  - **OLAP**
    - An engine tuned and architected for scanning and aggregating large volumes of data for analytics.
    - Optimized for column-oriented reads, compression, and throughput over large scans.
- **Data Warehouse**
  - An architecture pattern on an OLAP
    - a centralized repository of integrated, historical, structured data organized for analytical querying and reporting.
    - It implies integration from multiple source systems, schema modeling, and long-term historical retention.
    - OLAP is the engine class typically underneath it, but the warehouse is the data layer built on top.
  - **OLAP use cases that aren't a data warehouse**
    - **Customer-facing analytics**
      - sub-second dashboards for thousands of external users (your wedge).
    - **Real-time streaming**
      - ingest and query live event data as it lands.
    - **Observability backend**
      - powering logs, metrics, and monitoring platforms.
- **Warehouse vs. Customer-Facing Analytics** (different engine, not a bigger one)
  - Data Warehouse
    - Internal analysts (tens of users)
    - Complex ad-hoc queries
    - Sseconds-to-minutes latency OK.
  - Customer-Facing Analytics (opposite profile)
    - External users (thousands to millions)
    - Known/repeated queries
    - Sub-second latency required.
  - This flips the architecture, not just the dials
    - Customer Facing Analytics Requires
      - Pre-aggregation, materialized views, rollup tables, heavy caching, often different storage.
  - The failure mode (your wedge)
    - A warehouse "turned up to 11" (Snowflake, BigQuery) breaks on cost and concurrency under thousands of external users.
    - Hence a separate serving layer / different engine class: ClickHouse, Druid, Pinot, Vertica.
  - Takeaway
    - Same OLAP lineage, different design point. "Need an analytical engine" ≠ "need a warehouse."

Give it to me in bullet points

By the way. I want to be able to give you an instruction to make a bullet pointed list that has sub bullets, or sub sub bullets, or even sub sub sub, as many as is necessary. But I like a structured list just like the screenshot.

I don't want to tell you how many levels of sub bullets to use, whatever is most appropriate, and helps structure ideas and provide depth as makes it easiest when scanning information and creating understanding.

What would be a simple code I can give you when I want your answer done in that way.

Claude

Good code: **;struct** (for "structured").

Short, unlikely to collide with anything, and reads as what it does. When you append it, I'll return a nested bullet outline like your screenshot: as many sub-levels deep as the ideas warrant, no more, structured for fast scanning. It also pairs cleanly with your conciseness codes (e.g. `;struct ;2` = structured *and* tight).

Here's your data lake answer in that format:

- **Data Lake**
  - Raw files dumped into cheap object storage (S3, ADLS, GCS)
    - Open formats (Parquet, CSV, JSON)
    - No database engine on top
  - Core idea: store-everything-now, schema-later
    - Ingest anything (logs, events, images, semi-structured) without modeling up front
  - Strengths
    - Cheap
    - Flexible
  - Weaknesses
    - Slow (no indexes, stats, or engine-controlled layout)
    - Ungoverned (no ACID, no schema enforcement)
  - Add a table format on top (Iceberg, Delta, Hudi) → becomes a lakehouse

To lock the code in so you don't have to re-explain it each time, I can save it to memory. Want me to?

2026-07-10 09:19:27 **You Asked**

Why would someone want to schema later? ALso what is schema? ;2

Claude

Schema is the structure of your data: the tables, columns, data types, and rules defining what each field is (e.g. `user_id` is an integer, `email` is text, `signup_date` is a date). "Schema-on-write" (warehouses) forces you to define that structure before loading anything. "Schema-later" (schema-on-read, lakes) lets you dump raw data now and impose structure only when you query it. People want schema-later when they don't yet know how they'll use the data, when it arrives in messy/varied formats they don't want to model up front, when speed and cost of ingestion matter more than

immediate queryability, or when different consumers (ML, analytics) will each interpret the same raw data differently. The tradeoff is you defer the work, not eliminate it: someone still untangles that structure later, and if nobody does, you get a data swamp.

2026-07-10 09:19:50 **You Asked**

;struct

Claude

- **Schema**
  - The structure/definition of your data
    - Tables, columns, data types, rules
    - E.g. `user_id` = integer, `email` = text, `signup_date` = date
  - Two timing models
    - Schema-on-write (warehouses): define structure *before* loading
    - Schema-on-read / "schema-later" (lakes): dump raw now, impose structure *at query time*
- **Why schema-later**
  - Don't yet know how the data will be used
    - Defining structure early forces premature decisions
  - Data arrives messy or in varied formats
    - Avoid modeling it all up front
  - Ingestion speed/cost matters more than immediate queryability
    - Get it stored now, deal with structure later
  - Multiple consumers interpret the same raw data differently
    - ML, analytics, etc. each apply their own structure on read
- **The tradeoff**
  - Work is deferred, not eliminated
    - Someone still untangles the structure later
  - If nobody does → data swamp

2026-07-10 16:18:41 **You Asked**

So in Data Lakes, people just dump literally tables into cheap storage? ;2 (Make sure to check memory for that code as I've updated it)

Claude

Not usually literal database tables. The premise is slightly off. What lands in a lake is typically raw exported data: flat files (Parquet, CSV, JSON), log dumps, event streams, images, and semi-structured blobs.

Sometimes that raw data *originated* from a table (e.g. a nightly export of a production database), so it may resemble tabular rows. But it's dumped as files, stripped of the database engine, indexes, constraints, and live schema that made it a real "table."

So: files that may contain table-shaped data, but not tables in the operational sense. The table-ness (structure, types, queryability) gets reimposed on read, which is the schema-later point from before.

2026-07-10 16:26:14 **You Asked**

Parquet, CSV,
Are these not tables? ;1

Claude

Not quite. They're *files* that hold table-shaped data (rows and columns), but they lack what makes a real table: a live engine, indexes, constraints, and enforced schema. Table-*shaped*, not tables.

2026-07-10 16:26:55 **You Asked**

So you're saying when I create a table in Excel, it's not a table? That doesn't make sense to me.

Claude

Fair challenge. The word "table" is overloaded, and I was using it in a narrow database sense without saying so. Let me separate the meanings, because your Excel table absolutely is a table.

A **table** in the plain sense is any grid of rows and columns. Your Excel table qualifies. A CSV qualifies. A Parquet file qualifies. By this definition I was wrong to say they aren't tables.

A **table** in the strict database sense means that grid *plus* a live system managing it: enforced schema (every value in a column must be the declared type, or it's rejected), constraints (uniqueness, relationships between tables), indexes for fast lookup, and an engine handling concurrent reads/writes with ACID guarantees. This is what a warehouse or an OLTP database gives you.

A CSV or Parquet file has the grid but none of the managing system. Nothing stops a "date" column from containing the word "banana." There's no index, no constraint, no engine. That's the distinction I was pointing at: not "these aren't tables" but "these aren't *managed database tables*."

Where does Excel land? In between, and closer to your intuition than to a raw file. Excel enforces some types, supports formulas and relationships, and has a UI managing the grid. It's more than a CSV, less than a database. If you save that same Excel table *as* a CSV, you strip away Excel's management layer and keep only the grid, which is exactly what sitting in a data lake looks like.

So the accurate version of my earlier point: data lakes hold table-shaped files that carry the rows and columns but not the database engine around them. I overstated it by saying "not tables." Your pushback was right.

2026-07-10 16:32:58 **You Asked**

Give me a similar write up for what a data lakehouse is. Be sure to include what it's purpose is and what it must have to be considered a lake house (what was mentioned in the clickhouse article)

Keep it overall concise

* Data Lake
  * Raw files dumped into cheap object storage (S3, ADLS, GCS)
    * Open formats (Parquet, CSV, JSON)
    * No database engine on top
  * Core idea: store-everything-now, schema-later
    * Ingest anything (logs, events, images, semi-structured) without

modeling up front
  * Strengths
    * Cheap
    * Flexible
  * Weaknesses
    * Slow (no indexes, stats, or engine-controlled layout)
    * Ungoverned (no ACID, no schema enforcement)
  * Lake House
    * Add a table format on top (Iceberg, Delta, Hudi) → becomes a lakehouse

Claude

- **Data Lakehouse**
  - A data lake with a management layer added on top
    - Keeps the cheap object storage + open files
    - Adds back the database guarantees a raw lake lacked
  - Purpose
    - Get warehouse-grade reliability and governance *without* copying data into a separate warehouse
    - One copy of the data serves both worlds: cheap lake storage + trusted, queryable tables
  - What it must have (the four layers from the ClickHouse article)
    - Object storage — the cheap raw file storage (S3, ADLS, GCS)
    - Table format — Iceberg, Delta, or Hudi
      - This is the key ingredient that turns a lake into a lakehouse
      - Adds ACID, schema enforcement/evolution, snapshot isolation, time travel
    - Catalog — tracks which files make up each table
      - The single source of truth for what a "table" is
    - Query engine — reads/writes it all (Trino, Spark, Photon, etc.)
  - Net effect
    - Fixes the raw lake's two weaknesses: no longer ungoverned, no longer as slow
    - Still not a sub-second, high-concurrency serving engine (that tier stays separate)

2026-07-10 16:38:19 **You Asked**

So what came first, the data lake, or the data warehoure? Be concise

Claude

The data warehouse, by a wide margin.

- **Data warehouse**: concept formalized in the late 1980s–early 1990s (Bill Inmon, then Ralph Kimball).
- **Data lake**: term coined around 2010 (James Dixon), rising with Hadoop and cheap object storage.
- **Lakehouse**: newest, ~2020 (Databricks coined the term).

Warehouses came first because storage was expensive, so you modeled data carefully before loading it. Lakes emerged only once storage got cheap enough to dump everything first and structure later.

2026-07-10 16:38:34 **You Asked**

What did Hadoop do?

Claude

Hadoop was the open-source framework (mid-2000s, from Yahoo/Apache, based on Google's papers) that made cheap large-scale data storage and processing possible on clusters of ordinary servers. It's the technology that made data lakes practical in the first place.

Two core pieces:

- **HDFS (Hadoop Distributed File System)** — spread huge datasets across many cheap machines' disks, treating the cluster as one giant storage pool. Fault-tolerant: it replicated data so a dead server didn't lose anything. This is the "dump everything cheaply" storage that lakes were built on.

- **MapReduce** — a way to process that distributed data by shipping the computation *to* the machines holding the data, rather than pulling data to a central engine. You could crunch enormous datasets in parallel across the cluster.

Why it mattered: before Hadoop, scaling meant buying a bigger, expensive specialized machine (vertical scaling). Hadoop let you scale *horizontally* by adding more commodity boxes, which was dramatically cheaper. That cost collapse is what made "store everything now, structure later" economically viable, and the data lake concept emerged directly on top of it.

Worth noting: Hadoop itself is now largely legacy. The compute side (MapReduce) was slow and clunky, and got displaced by Spark and then by cloud object storage (S3) replacing HDFS. But the lake idea it enabled outlived the technology. Modern lakes and lakehouses kept the "cheap distributed storage + open files" premise and swapped Hadoop's guts for faster, cloud-native pieces.

2026-07-10 16:43:37 **You Asked**

So there are a lot of tools I always hear about. Spark. Hadoop. I never know what they are. Concisely. What are the main tools in this space I should know of. ;Struct

Claude

- **Storage layer** (where data physically sits)
  - **HDFS** — Hadoop's distributed file system; original cheap cluster storage, now legacy
  - **S3 / ADLS / GCS** — cloud object storage; replaced HDFS as the default lake storage
- **Table formats** (turn raw files into governed tables → the lakehouse layer)
  - **Apache Iceberg** — most open, broadest engine support; current momentum leader
  - **Delta Lake** — Databricks' format
  - **Apache Hudi** — strong at streaming/upserts (CDC)
- **Catalogs** (track what tables exist, the source of truth)
  - **Hive Metastore** — the old standard
  - **AWS Glue, Unity Catalog (Databricks), Polaris** — modern successors
- **Compute / query engines** (process the data)
  - **Hadoop MapReduce** — original batch engine; slow, now legacy

    - **Apache Spark** — replaced MapReduce; general-purpose batch + ML + streaming; the dominant workhorse
    - **Trino / Presto** — fast distributed SQL query engine over lakes
    - **Databricks Photon** — Databricks' fast SQL engine on Delta
- **Serving engines** (sub-second, high-concurrency — the separate hot tier)
    - **ClickHouse, Druid, Pinot** — customer-facing analytics
    - **Vertica** — engine-controlled format + hot local tier (your wedge)
- **Streaming** (data in motion, feeds the lake)
    - **Apache Kafka** — the standard event/streaming backbone
- **Orchestration** (schedules the pipelines)
    - **Apache Airflow** — the standard job scheduler/DAG tool
    - **dbt** — SQL-based transformation/modeling layer

Mental model: **storage → table format → catalog → compute → (serving)**, with Kafka feeding data in and Airflow/dbt coordinating the flow. Spark and Kafka are the two names you'll hear most; both predate the lakehouse and remain everywhere.

2026-07-10 16:45:41 **You Asked**

Lakehouse: newest, ~2020 (Databricks coined the term).
So DataBricks is built on this concept of data lakes?

First answer that. BUt then if so, why do they still have prorprietary format?

Be concise

Claude

Yes, Databricks is built on the lake/lakehouse concept. In fact they coined the term "lakehouse." Their whole architecture is open files (Delta/Parquet) on cheap object storage (S3/ADLS/GCS), with a table format and engine layered on top. That *is* the lakehouse pattern.

On the "proprietary format" point, one clarification: Delta Lake itself is open-sourced, not proprietary. The format spec is public and other engines can read it. So it's not proprietary the way Vertica's format is.

But your instinct is still pointing at something real. Two things are genuinely more closed in the Databricks world:

- **Practical openness vs. spec openness.** Delta is open on paper, but its richest features and best performance have historically landed first (and worked best) inside Databricks' own engine. Open-source Delta lagged the version Databricks ran internally. So "open" was partly real, partly a funnel back to their platform.
- **Unity Catalog and Photon.** The catalog (governance/metadata) and Photon (their fast query engine) are the actually-proprietary pieces. That's where the lock-in lives, not in the file format.

So the answer to your "why still proprietary": the *format* isn't the moat, and keeping it open was strategic. It won adoption and made Databricks the center of gravity for the lakehouse. They monetize the layers *around* the open format, the engine, catalog, and managed platform, which is where proprietary control actually pays off. Open the format, close the engine.

2026-07-10 16:49:04 **You Asked**

Ok so it's an open format, that can work anywhere, but works best inside of Data Bricks? ;1

Claude

Exactly. Delta is genuinely open and readable by other engines, but its newest features and best performance historically land first and run best inside Databricks. Open enough to adopt freely, tuned enough to pull you toward their platform.

2026-07-10 16:51:22 **You Asked**

So concisely, what's the difference between Iceberg, Delta, ROS, snowflake's format?

Like I am not asking you to provide a detailed comparision of each.

I'm just trying to understand fundamentally why we have different table formats to begin with.

;2 (Please follow the code and prose accordingly)

Claude

Fundamentally, they exist because different vendors each needed to solve the same problem, add database guarantees (ACID, schema, versioning) on top of raw files, and each built their own version to fit their engine and their strategy.

The deeper split is *how open* they are. Iceberg and Delta are open table formats meant to sit on your cheap storage and be read by many engines, born from the lakehouse goal of decoupling storage from compute. They differ mostly in origin and details (Iceberg from Netflix, vendor-neutral; Delta from Databricks, tied more to its ecosystem), not in core purpose.

Vertica's ROS and Snowflake's format are a different category: proprietary, engine-controlled layouts baked into the product. They aren't trying to be open or portable. They trade interoperability for speed and tight integration, because the vendor controls both the format and the engine reading it.

So the real axis isn't Iceberg-vs-Delta-vs-the-rest on features. It's *open interoperable formats* (many engines, portable, lakehouse philosophy) versus *proprietary engine-owned formats* (one engine, faster, lock-in). Multiple names exist mostly because several vendors raced to plant their flag on the same idea.

2026-07-10 16:55:26 **You Asked**

How is Delta open, but ROS isn't?

Like, other companies don't have the right to run ROS?

;1

Claude

Right. "Open" means the format spec is published and licensed so anyone can build software that reads/writes it. Delta's spec is public; other engines legally and technically can implement it. ROS is

proprietary and undocumented publicly, so only Vertica's engine can read it. No one else has the spec or the right to use it.

2026-07-10 16:56:34 **You Asked**

What other engines use Delta ? ;1

Claude

Apache Spark (native/primary), plus Trino, Presto, Apache Flink, and via connectors Snowflake, BigQuery, Athena, and Redshift can read Delta tables. Reads are broadly supported; full write support is narrower, strongest in Spark.

2026-07-10 16:57:42 **You Asked**

Wait i thought Delta came from DataBricks, not Apache Spark.

I'm not understanding.

Also, what's Apache? What's spark

;2

Claude

Both facts are true, and the confusion is understandable: Databricks created Delta, but Databricks also created Spark. The company built both, then donated Spark (and later Delta) to open source. So "Delta came from Databricks" and "Delta runs natively on Spark" aren't in tension. They came from the same place.

Let me untangle the names.

Apache isn't a product. It's the Apache Software Foundation, a nonprofit that stewards open-source projects. When a company donates a project to it, the project gets the "Apache" prefix as a signal that it's community-governed and not owned by one vendor. Hence Apache Spark, Apache Iceberg, Apache

Kafka, Apache Hadoop. It's a label for "open-source project held in a neutral foundation," not a company or a tool.

Spark is a compute/query engine: the thing that actually processes data (runs the queries, transformations, ML jobs) across a cluster. It replaced Hadoop's slow MapReduce and became the dominant workhorse for large-scale data processing. Databricks was founded by Spark's creators, which is why Databricks, Spark, and Delta are all intertwined: same people, same origin. Databricks commercializes the open engine and adds its proprietary layers (Photon, Unity Catalog) on top.

2026-07-10 17:02:17 **You Asked**

So DataBricks was founded by Sparks creators.

Why did they create spark to begin with? Who's paying these dudes to just make random shit? What am I not seeing here? ;2

Claude

Nobody was paying them at first. Spark started around 2009 as an academic research project at UC Berkeley's AMPLab, a university research lab. The creators (Matei Zaharia and others) were grad students and academics. Their motivation was research, not product: Hadoop's MapReduce was painfully slow for iterative work like machine learning, and they built Spark to fix that specific technical problem. University labs run on grants (government, corporate sponsorships), so that's the "who paid" early on. Cheap, indirect, not tied to a product.

The move you're sensing comes next. Once Spark got popular as free open-source software, the creators did what many successful open-source authors do: they founded a company (Databricks, 2013) to commercialize it. The logic is that the free tool creates massive adoption, and adoption creates a market for the thing companies actually pay for: a managed, hosted, easier-to-run version plus proprietary add-ons (Photon, Unity Catalog) and enterprise support. Give away the engine, sell the platform around it.

So what you're not seeing is the standard open-source playbook: research lab births the tech for free → it goes open source and spreads → creators monetize the ecosystem that grew around it. The "random free shit" is the customer-acquisition funnel. The proprietary layers are the business.

2026-07-10 17:19:41 **You Asked**

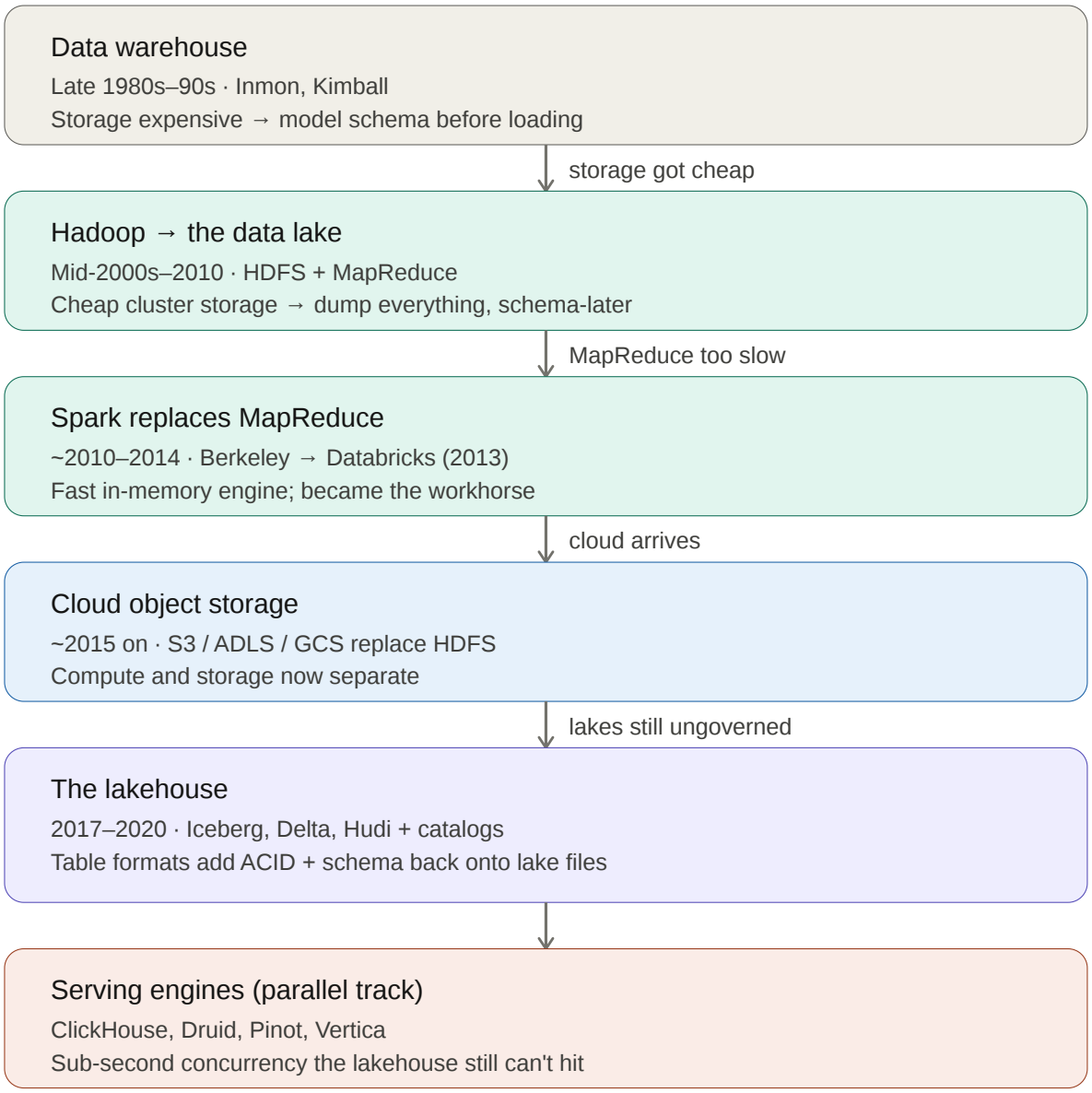
Ok, so the learning the history of all this stuff is actually helping me understand the space a ton.

Give me a history of the space, the tools, etc.

Claude

I'll build this as a walkthrough diagram of the timeline, then give you the narrative. Let me lay out the evolution visually first, since the eras and how each tool triggered the next is the thing worth seeing at a glance.

Here's the timeline as a diagram. The key thing to see is how each era was *triggered* by a cost or performance problem in the one before it.



Now the narrative behind it. The whole space is a chain of reactions to one recurring problem: storage and compute kept getting cheaper, and each cost collapse unlocked a new way of working.

The **warehouse era (late 1980s–90s)** started because storage was genuinely expensive. When every gigabyte costs real money, you can't afford to store junk, so you carefully model your data first (schema-on-write) and load only clean, structured, business-ready tables. Bill Inmon and Ralph Kimball wrote the foundational playbooks. Teradata, later Oracle and Netezza, were the machines. The defining trait: expensive, rigid, trustworthy, and you scaled by buying a bigger box.

The **Hadoop / data lake era (mid-2000s–2010)** happened because the web giants (Google, Yahoo, Facebook) generated more data than any single expensive box could hold. Google published papers describing how they spread data and computation across thousands of cheap commodity servers. Yahoo built the open-source version, Hadoop, with HDFS for storage and MapReduce for processing. Suddenly storage was cheap enough to dump everything first and structure later. That economic shift is what birthed the data lake concept (term coined ~2010).

The **Spark era (~2010–2014)** was a direct reaction to MapReduce being painfully slow and clumsy, especially for iterative work like machine learning. Researchers at UC Berkeley built Spark to fix it: an in-memory engine that ran the same distributed jobs far faster. It went open source, spread rapidly, and its creators founded Databricks in 2013 to commercialize it. Spark displaced MapReduce and became the default processing engine for large-scale data.

The **cloud era (~2015 on)** replaced the storage half of Hadoop. Amazon's S3 (and Azure's ADLS, Google's GCS) offered effectively infinite, cheap, managed object storage with no clusters to babysit. Lakes moved off HDFS onto S3. This is also where "separate compute from storage" became the norm: your data sits in S3, your engines spin up and down independently against it.

The **lakehouse era (2017–2020)** addressed the lake's original sin: it was cheap and flexible but ungoverned and slow, the data swamp problem. The fix was the table format. Netflix built Iceberg, Databricks built Delta, Uber built Hudi. Each adds a metadata layer on top of the raw files that brings back ACID transactions, schema enforcement, and versioning, turning a pile of files into something that behaves like real tables. Databricks coined "lakehouse" (~2020) to name the result: lake economics plus warehouse guarantees on one copy of the data.

Running **alongside all of this** is the serving-engine track, which solved a different problem the mainstream lineage never did: sub-second queries for thousands of concurrent users. ClickHouse (Yandex), Druid, and Pinot (LinkedIn) emerged from companies needing customer-facing analytics at scale. Vertica predates most of them (mid-2000s, from database pioneer Michael Stonebraker) with the same bet: an engine-controlled columnar format plus a hot local tier. This is the track your product lives on, and it stayed separate precisely because openness and general-purpose layout can't deliver that latency profile.

The throughline worth remembering: **warehouse → lake → lakehouse is one storyline** driven by falling storage cost and the tension between "governed but rigid" and "cheap but wild." **Serving**

engines are a parallel storyline driven by concurrency and latency, not cost. They never merged, which is exactly why the two-tier "lakehouse for cold, native engine for hot" pattern exists today.

Want me to save a condensed version of this history to memory so it's on hand for your positioning work, or go deeper on any single era?

2026-07-10 17:22:33 **You Asked**

What's wrong with ungoverned? ;1

Claude

Ungoverned means no ACID (corrupt/half-written data), no schema enforcement (a "date" column can hold "banana"), and no single source of truth. So you can't trust the data is correct, complete, or consistent, and you can't reliably control access, audit it, or track lineage. Untrusted data is useless for decisions.

2026-07-10 17:23:55 **You Asked**

So what was before the data warehouse? ;2

Claude

Before the warehouse, businesses ran on operational databases (OLTP systems), the transactional databases that ran day-to-day operations: orders, inventory, payroll, banking records. These were built to record transactions fast, not to analyze them. Before those, in the 1960s–70s, it was mainframes with flat files and hierarchical/network databases (IBM's IMS, then later relational databases after Edgar Codd's 1970 relational model and the rise of SQL).

The problem that created the warehouse: when analysts tried to run big reporting queries directly on these operational databases, they slowed down or competed with the live business transactions, and each system was a separate silo (sales in one, inventory in another) with no unified view. Companies also kept little history, since operational systems overwrite old data. The warehouse was invented to solve exactly that: copy data out of all those operational systems into one separate place, integrated and keeping full history, so analysts could run heavy queries without disturbing the live systems.

That's the "integration from multiple source systems" and "long-term historical retention" in your outline.

2026-07-10 17:26:23 **You Asked**

Before those, in the 1960s–70s, it was mainframes with flat files and hierarchical/network databases (IBM's IMS, then later relational databases after Edgar Codd's 1970 relational model and the rise of SQL).
What do you mean? ;2

Claude

I mean that before modern databases existed, computers still had to store business data, but the tools were far more primitive. In the 1960s–70s, computing ran on mainframes (giant central computers), and data was stored in two main ways.

First, flat files: data written to tape or disk as plain records with no relationships between them, just sequential lists you had to read top to bottom. Second, early "navigational" databases (hierarchical and network models, like IBM's IMS) where records were physically linked in tree or web structures, and to find data you had to manually follow those links, pointer by pointer, in your program code. Both were rigid: the way data was physically stored was welded to how you accessed it, so a new kind of question often meant rewriting code or restructuring the data.

The breakthrough was Edgar Codd (an IBM researcher) publishing the relational model in 1970. His idea: store data as simple tables and let people query it by *describing what they want* (via a language that became SQL) rather than manually navigating pointers. That decoupled the question from the physical storage, which is why relational databases (and SQL) took over in the 1980s and became the foundation everything since, including warehouses, is built on.

2026-07-10 17:56:02 **You Asked**

Thanks

Claude

You're welcome. This was a good thread. You built up the whole space from first principles: warehouse vs. lake vs. lakehouse, the open-vs-proprietary format axis, and the history that explains why it all looks the way it does. That grounding maps directly onto where Vertica sits and why the serving tier stayed separate.

Ping me anytime you want to go deeper or pressure-test the positioning.

2026-07-10 17:57:33 **You Asked**

First, flat files: data written to tape or disk as plain records with no relationships between them, just sequential lists you had to read top to bottom.
So was this just a standard like Excel table?

Claude

Not really, and the difference is instructive.

A flat file was just raw records stored in sequence, with no software managing or displaying them. Picture a plain text file where each line is a record and fields are separated by commas or fixed positions: `1001,Smith,Portland,4500`. No grid, no cells, no formulas, no interface. To do anything with it, you wrote a program to read it line by line. The "flatness" means no relationships and no structure beyond the raw sequence, and critically, nothing enforcing what each field contains.

Excel is the opposite in almost every way: a live application with a visual grid, typed cells, formulas, sorting, and a management layer sitting on top of the data. It's actively helping you.

The closer modern analogy to a flat file is a CSV file, which is exactly what you get if you export that Excel sheet and strip away Excel itself. Same rows and columns of data, but now just inert text with no app around it. That's the connection back to what we discussed earlier: a raw file in a data lake is essentially a flat file (often CSV or Parquet). Table-*shaped* data with no engine managing it. So flat files from the 1960s and lake files today are the same fundamental idea, just cheaper and at massive scale now.

2026-07-10 18:00:35 **You Asked**

Ok, so the original way information was stored was a plain text file, where each line is a record, and fields are seperates by commas.

An excel table just provide essentially a user friendly way to view that info?

;1

Claude

Roughly, but Excel does more than *view* it. It also stores, types, and computes: formulas, sorting, enforced cell types, a calculation engine. A pure viewer just displays. Excel is a viewing layer *plus* a lightweight management/compute layer on top of the underlying rows and columns.

2026-07-10 18:01:26 You Asked

Ok really concisely, what came before those plain text files? And what came after them? Just bullet point each step. Be concise.

Claude

- **Before**
  - Punch cards (holes in stiff paper = data), 1890s–1960s
  - Magnetic tape (sequential records on reels)
- **Plain flat files** (sequential records on tape/disk)
- **After**
  - Hierarchical/network databases (records linked by pointers) — IBM IMS, 1960s–70s
  - Relational databases + SQL (data as tables, queried by description) — after Codd, 1970s–80s
  - Data warehouses (integrated analytical databases) — late 1980s–90s
  - Data lakes (cheap raw file storage at scale) — ~2010
  - Lakehouses (table formats add governance back) — ~2020

2026-07-10 18:02:43 **You Asked**

Punch cards?

Magnetic tape?

We are talking about data management for records, correct?

Be concise

Claude

Yes, same lineage: physically storing and retrieving business records, just on older media.

- **Punch cards** — a card with holes punched in specific positions; each hole pattern encoded a number or letter. One card typically held one record (say, one employee or one census entry). Machines read the holes mechanically to tabulate and sort. This is literally how the 1890 US Census and early payroll/accounting were processed, decades before computers.
- **Magnetic tape** — reels of the same tape idea as cassettes, storing records as magnetic patterns. Replaced cards because it held far more and was faster to read. But it was *sequential*: to reach record 900 you spooled through the first 899. That sequential nature is exactly what "flat file" inherited.

So the throughline: punch cards → tape → flat files on disk → databases. Each step stored the same kind of records, just faster to search and easier to relate to each other.

2026-07-10 18:03:56 **You Asked**

So at the core of data management is essentially the storage of records? ;1

Claude

Yes. At its core it's storing records so they can be reliably retrieved, related, and queried later. Everything since (databases, warehouses, lakes) is just better ways to store, organize, and ask questions of those records at ever-larger scale.

2026-07-10 18:04:25 **You Asked**

What are records? ;1

Claude

A record is the data about one thing: one customer, one order, one transaction. It's a row, a bundle of related fields (e.g. name, address, balance) describing a single entity. Records are the basic unit data management stores and retrieves.

2026-07-11 04:25:07 **You Asked**

Hierarchical/network databases (records linked by pointers) — IBM IMS, 1960s–70s
Relational databases + SQL (data as tables, queried by description) — after Codd, 1970s–80s
Data warehouses (integrated analytical databases) — late 1980s–90s
What's the difference between each of these?

Claude

Each solved a different problem, and they build on each other rather than replacing one another entirely.

Hierarchical/network databases were the first attempt to move beyond dumb sequential files. They stored records and physically linked them with pointers, like a tree (each record has one parent) or a web (records can have multiple links). To get data, your program had to *navigate* those links manually: start here, follow this pointer, then that one. It was fast if you knew the path, but brittle. The way data was physically stored was welded to how you queried it, so a new question often meant rewriting code. You could only ask questions the designer anticipated.

Relational databases (Codd's 1970 idea, commercialized in the 80s) broke that welding. Data lives in simple tables, and instead of navigating pointers, you *describe what you want* in SQL ("give me all customers in Oregon with a balance over $500") and the database figures out how to get it. That decoupling of the question from the physical storage is the single biggest leap in this history. Suddenly you could ask questions nobody planned for, without touching the storage.

Data warehouses aren't a new kind of database. They're an *architecture pattern built on relational databases*. The problem they solved wasn't "how do we store or query data," it was "our operational databases are choking." Analysts running heavy reports on live transactional systems slowed down the actual business, and the data was scattered across separate silos (sales here, inventory there) with no history kept. So the warehouse copies data out of all those systems into one separate place, integrates it, and retains full history. Same relational technology underneath, but a different *purpose*: analysis rather than operations.

So the arc is: hierarchical solved *linking* records, relational solved *querying* them flexibly, and the warehouse solved *where to put them* for analysis without hurting the live business.

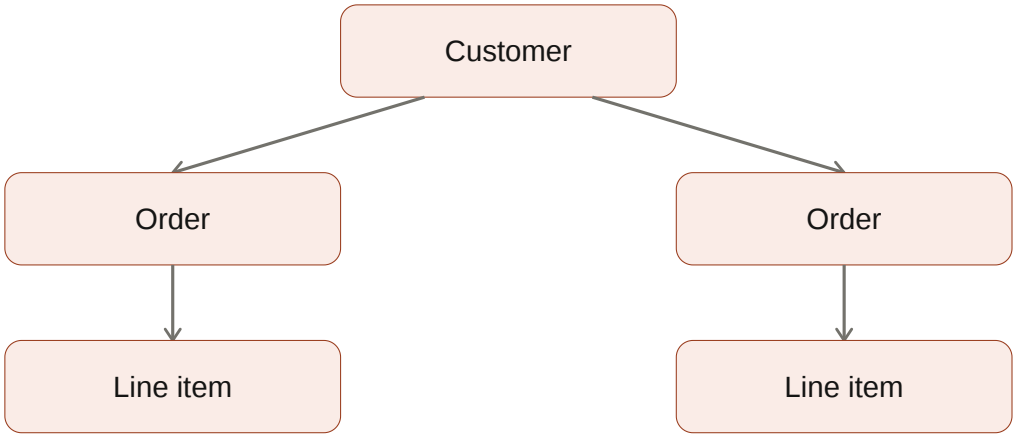
2026-07-11 04:27:30 **You Asked**

Can you visually give me difference between hierarchial/network & relational?

Claude

Here's the contrast. Watch what happens to the *arrows*: they're the entire difference.

Hierarchical / network — follow the pointers



Links are physical. Your code walks them step by step.

Relational — match the values

No pointers. Tables stand alone; keys connect them.

You describe the result in SQL; the engine finds the path.

The top half is a **map you must walk**. The records are physically chained together, so to find a customer's line items your program starts at Customer, follows the pointer to Order, then follows another to Line item. If the boss asks a question that runs *against* the arrows ("which customers bought product X?"), you're stuck: there's no pointer pointing that way. You either rewrite the code or restructure the data.

The bottom half is a **set of independent tables**. Nothing is physically linked. `Orders` just carries a `customer_id` value, and `Line items` carries an `order_id`. The connection is *logical*, a matching value, not a hardwired pointer. So you write `SELECT ... WHERE city = 'Portland'` and the engine works out how to join them. Any question is answerable in any direction, and the query optimizer figures out the fastest path for you.

That's the whole leap. Hierarchical welds the *question* to the *storage*. Relational separates them. Every flexible ad-hoc query you've ever run exists because of that split.

2026-07-11 04:29:05 **You Asked**

("which customers bought product X?" Why couldn't you ask that?

Claude

Because in the hierarchical model, the pointers only run **one direction**: Customer → Order → Line item. That's the only path that physically exists.

To ask "which customers bought product X," you need to go the *other* way: start at the product, find the line items containing it, then walk back up to the orders, then back up to the customers. But there is no pointer from a line item back to its order, or from a product to the line items that reference it. Those links were never built.

So your only option was brute force: start at customer #1, walk down every one of their orders, check every line item, see if product X is there, then repeat for all million customers. Wildly slow, and you'd have to hand-write that traversal code yourself.

The alternative was to *pre-build* the reverse structure: physically create a Product → Line item chain in advance. But that meant anticipating the question ahead of time, restructuring the data, and maintaining another set of pointers. Every unanticipated question meant more engineering work.

Relational kills this problem because there are no directional pointers to begin with. `Line items` carries an `order_id` and a `product_id` as plain values. You can match on *any* column in *any* direction, so "which customers bought product X" is just as easy as "which products did customer Y buy." The question is no longer constrained by how the data was physically wired.

2026-07-11 04:31:08 **You Asked**

So concisely, before hierarchical/network, it was just dumb sequential files? Did people querry them for analytics? Could they update them like a OLTP?

Claude

Yes, before hierarchical/network it was just flat sequential files (tape and early disk).

Could they query them for analytics? Sort of, but nothing like today. There was no query language and no ad-hoc questioning. To get any answer, a programmer wrote a custom program that read the file from start to finish, filtered, and tallied. Every new question meant new code, and it ran as an overnight batch job. So "analytics" existed, but it was slow, rigid, and mediated entirely by programmers.

Could they update them like OLTP? No, not really, and that's the key limitation. Tape is sequential-only: you can't jump to record 900 and change it in place. The standard pattern was batch update: read the whole old tape, merge in a batch of changes, write out an entirely new tape. That means no random access, no real-time updates, and no concurrent transactions. Nothing like an OLTP database.

That limitation is exactly what drove the next step. Disk storage enabled random access (jump straight to a record), and hierarchical databases like IBM's IMS were built on top of that to allow fast, direct record lookup and update. That made real transactional processing possible for the first time.

2026-07-11 04:32:18 **You Asked**

Does random access refer to the ability to read & edit any point in a files storage? ;2

Claude

Yes, exactly. Random access means you can jump directly to any location in storage and read or write it, without traversing everything before it. "Random" here means "arbitrary," not "unpredictable": any address, on demand, in roughly the same amount of time.

The contrast is sequential access, which is what tape forced. To reach record 900 you physically spool through records 1 through 899. Disk changed that: the read head can move straight to the location holding record 900. That single capability is what made transactional databases possible, since updating one customer's balance in place requires reaching that one record directly rather than rewriting the entire dataset.

2026-07-11 05:26:02 **You Asked**

I mean that before modern databases existed, computers still had to store business data, but the tools were far more primitive. In the

1960s–70s, computing ran on mainframes (giant central computers), and data was stored in two main ways.
Is this where IBM was in it's hayday? ;2

Claude

Yes, that was IBM's peak dominance. The System/360 (launched 1964) essentially defined enterprise computing, and IBM controlled roughly 70% of the mainframe market. The saying "nobody ever got fired for buying IBM" comes from exactly this era. They faced a landmark antitrust suit from the US government (filed 1969, dropped 1982) precisely because they were so dominant.

Worth noting the irony in your thread: IBM also employed Edgar Codd, whose 1970 relational model eventually undermined IBM's own hierarchical IMS product. IBM was slow to commercialize it (they were protecting IMS revenue), which let Larry Ellison read Codd's paper and build Oracle first. Classic incumbent's dilemma, and a decent reminder that owning the research doesn't mean owning the market.

2026-07-11 05:36:42 **You Asked**

What's difference between Spark & Vertica ;1

Claude

Spark is a general-purpose *processing* engine (batch, ETL, ML, streaming) that reads open files it doesn't own. Vertica is a *database*: it owns its storage format, enforces schema, and is built for fast SQL analytics. Spark transforms data; Vertica stores and serves it.

2026-07-11 05:37:07 **You Asked**

Isn't Vertica a processing engine?

Claude

Fair correction, and you're right to push. Vertica *does* process data. It has a query engine that parses, plans, optimizes, and executes SQL. Saying "Spark transforms, Vertica stores" was too clean a split and understated Vertica's compute side.

The accurate framing: **Vertica is a full database (storage engine + query engine, tightly coupled). Spark is only a query/processing engine, with no storage of its own.**

That's the real difference. Vertica owns both halves, so it controls how data is physically laid out (projections, sort orders, encoding) and can optimize execution against a layout it designed. Spark has no storage layer at all. It reads whatever files you point it at (Parquet on S3, Delta, whatever) and processes them. It's a compute engine that borrows other people's storage.

The consequence flows directly from that: Vertica can hit sub-second analytics because it controls the layout its engine reads. Spark can't, because it's always reading a format it didn't design. Conversely, Spark is far more flexible: it does ETL, machine learning, streaming, and arbitrary code in Python/Scala, not just SQL. Vertica is SQL analytics, deeply optimized.

So both process. Only one owns storage.

2026-07-11 05:38:44 **You Asked**

To ensure we are on same page, Vertica is two pieces, a database and a processing engine ;1

Claude

Close, but one adjustment: **a database *is* those two pieces.** Vertica is a database, and a database = storage engine + query (processing) engine.

So it's not "a database *and* a processing engine." It's a database, which contains a storage engine and a processing engine. Spark is only the second half.

2026-07-11 05:39:13 **You Asked**

So Spark is purely a processing engine? ;1

Claude

Yes. Spark is purely a processing/compute engine, no storage of its own. It reads files from wherever they live (S3, HDFS, etc.) and processes them.